

The First Step of Dobbertin’s Theorem 1, Formalised

A map of the DobbertinStep1 MVP: paper $\leftrightarrow$ Lean

Abstract

This note maps the Lean 4 / Mathlib formalisation in the `DobbertinStep1` library onto the opening step of the proof of Theorem 1 in H. Dobbertin, *Kasami Power Functions, Permutation Polynomials and Cyclic Difference Sets* (NATO Sci. Ser. C **542**, 1999, pp. 133–158): the implication *equation (1)* $\Rightarrow$ *equation (2)*. We give the paper $\leftrightarrow$ Lean dictionary, a commutative diagram of the derivation, the lemma- and module-level dependency DAGs, and the `SimpleGraph` encoding of the proof space (proved connected in Lean).

1 The step in the paper

Let $L = \mathbb{F}_{2^n}$, $\gcd(k, n) = 1$, $k < n$, and $kk' \equiv 1 \pmod{n}$. Dobbertin studies, for fixed $c \in L$, the equation

$$cx^{2^k+1} = \sum_{i=1}^{k'} x^{2^{ik}} + \alpha \operatorname{Tr}(x), \quad (1)$$

and derives from it the *linearized* equation

$$\ell(x) = c^{2^k} x^{2^{2k}} + x^{2^k} + cx + 1 = 0. \quad (2)$$

“...we shall show that for each fixed $c \in L$, the equation (1) has at most one solution in L . Adding the 2^k -th power of this equation to itself we get ... and consequently $\ell(x) = c^{2^k} x^{2^{2k}} + x^{2^k} + cx + 1 = 0$. Note that $\ell(x) = 0$ if and only if $cx^{2^k+1} + \sum_{i=1}^{k'} x^{2^{ik}} + \alpha \operatorname{Tr}(x) = 0$ or 1.”

The Lean proof realises “adding the 2^k -th power to itself” by an Artin–Schreier *telescoping* of a partial trace, and the “ $= 0$ or 1” by the fact that $\operatorname{Tr}(x)$ is a *bit*. A genuine hypothesis $x \neq 0$ is needed: at $x = 0$ equation (1) holds vacuously ($0 = 0$) while $\ell(0) = 1 \neq 0$.

2 Paper $\leftrightarrow$ Lean dictionary

Paper	Meaning	Lean (<code>Dobbertin.Step1</code>)
$\operatorname{Tr}(x) = \sum_{i < n} x^{2^i}$	absolute trace	<code>trace n x</code>
$S(x) = \sum_{i=1}^{k'} x^{2^{ik}}$	numerator sum	<code>numeratorSum k k' x</code>
$P(x) = \sum_{j < k'} x^{2^{jk}}$	partial trace	<code>partialTrace k k' x</code>
eq. (1)	cleared equation	<code>equation1 n k k' α c x</code>
$\ell(x)$ of eq. (2)	linearized polynomial	<code>linearized k c x</code>
$S = P^{2^k}$	Frobenius image	<code>numeratorSum_eq_partialTrace_frob</code>
$P^{2^k} + P = x^2 + x$	Artin–Schreier	<code>partialTrace_telescope</code>
$x^{2^r} = x^{2^{r \bmod n}}$	Frobenius period	<code>pow_two_pow_mod</code>
$\operatorname{Tr}(x) \in \{0, 1\}$	the trace is a bit	<code>trace_eq_zero_or_one</code>
$(1) \wedge x \neq 0 \Rightarrow \ell = 0$	the derivation	<code>linearized_eq_zero_of_solution</code>
(1)$\Rightarrow$(2)	headline	<code>equation2_of_equation1</code>

3 The derivation as a commutative diagram

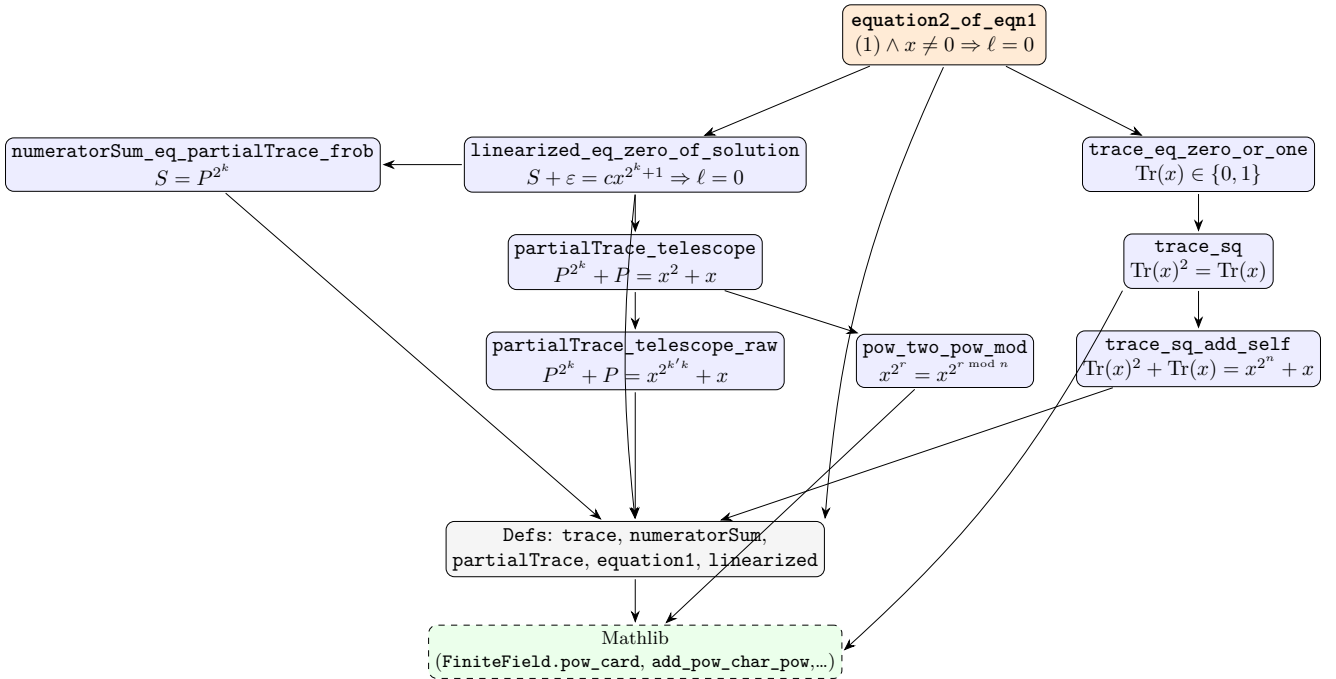
Write $\varepsilon = \alpha \text{Tr}(x) \in \{0, 1\}$. The two inputs (telescoping and the trace bit) combine into the single implication $(1) \Rightarrow (2)$:

$$\begin{array}{ccc}
 \boxed{cx^{2^k+1} = S(x) + \alpha \text{Tr}(x)} & \xrightarrow{\text{e_eq_zero_or_one}} & S(x) + \varepsilon = cx^{2^k+1}, \varepsilon \in \{0, 1\} \\
 \downarrow \text{collapse } \alpha \text{Tr to a bit} & & \downarrow \begin{array}{l} S=P^{2^k} \\ P^{2^k}+P=x^2+x \end{array} \\
 \boxed{\ell(x) = 0} & \xleftarrow[\text{divide by } x^{2^k} \neq 0]{\text{raise to } 2^k, \varepsilon^{2^k} = \varepsilon} & P = (x^2 + x) + cx^{2^k+1} + \varepsilon
 \end{array}$$

The right-hand column is `linearized_eq_zero_of_solution`; the top edge turns $\alpha \text{Tr}(x)$ into a bit ε ; the composite is the headline `equation2_of_equation1`.

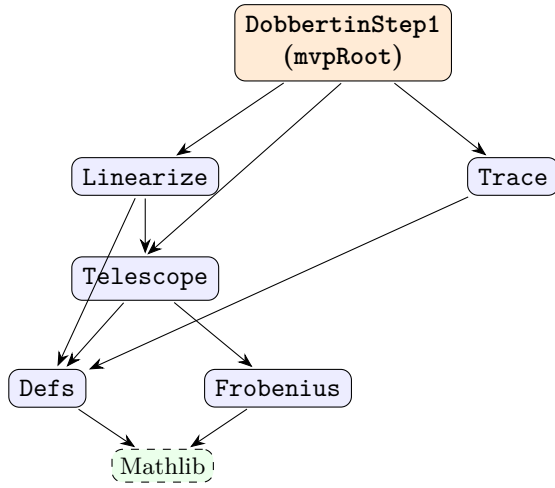
4 Lemma-level dependency DAG

Edge $A \rightarrow B$ means “ A is proved using B ”.

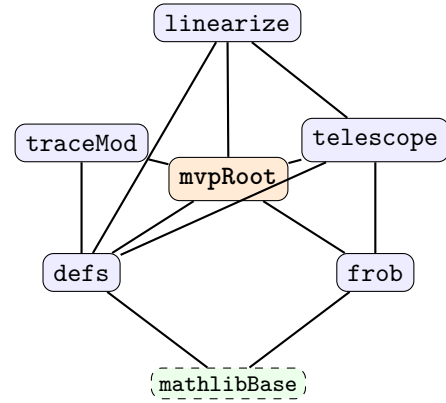


5 Module-level DAG and the SimpleGraph proof space

The library is split into six modules over a common Mathlib base. The directed DAG (left) is transcribed *verbatim* into the Lean relation `dep`; its undirected SimpleGraph `depGraph` (right) is proved **connected**.



directed DAG = Lean dep



undirected depGraph = SimpleGraph.fromRel dep

In DobbertinStep1/ProofSpace.lean:

```
inductive Node
| mathlibBase | defs | frob | traceMod | telescope | linearize | mvpRoot

def dep : Node → Node → Prop ...
def depGraph : SimpleGraph Node := SimpleGraph.fromRel dep

theorem reach_mathlibBase (v : Node) : depGraph.Reachable mathlibBase v
theorem depGraph_connected : depGraph.Connected
```

The theorem `depGraph_connected` certifies that the entire proof space collapses, through its dependency edges, onto the single `mathlibBase` node: there is no orphaned or free-floating component.

Soundness

Every declaration in `DobbertinStep1` is `sorry`-free; the headline `equation2_of_equation1` depends only on the standard axioms `propext`, `Classical.choice`, `Quot.sound`.